

图形学物理仿真 Tutorial for Lab 3

概览

在 Lab 3 中，你将实现一个基于有限元方法（Finite Element Method, FEM）的物理仿真框架。基础要求为实现一个：显式时间积分 + 线性有限元模型。

本次实验的核心目标包括：

- 构建一个三维软体仿真系统
- 基于线性有限元模型计算内力
- 实现用户交互驱动的动态响应

Demo 要求

软体模拟（必做）

- 基于线性有限元方法实现三维空间中的软体仿真模拟。
- 支持基本物理属性（弹性、质量、重力等）
- 实现用户交互，通过鼠标或键盘给软体施加外力。

Bonus 进阶挑战（任选，3次lab的Bonus得分上限为16分）

B1. 多种超弹性模型对比 ★☆☆

- 至少实现 3 种模型（如 StVK、Neo-Hookean、Corotated）
- 支持用户交互，展示差异

B2. 布料模拟 ★★☆☆

- 基于有限元方法实现一个三维空间中的布料模拟。
- 实现用户交互，通过鼠标或键盘给布料施加外力。
- 注意：布料是二维流形嵌入三维空间，deformation gradient 为 3×2

B3. 软体与环境的简单碰撞检测与响应 ★★☆☆

- 实现软体与解析刚体之间的简单碰撞检测，如：y=0的地面，x=0的墙面），球体，AABB长方体等；
- 实现基于Penalty 方法的碰撞解除（检测到碰撞后，基于最近点距离，施加一个解除碰撞的弹簧力）

B4. 实现隐式的仿真 ★★★

- 实现隐式时间积分（Implicit FEM）
- 可使用：牛顿迭代 和 共轭梯度（CG），可调包。
- 不要求高效率，但要求正确性

B5. 弹塑性与粘弹性材料 ★★★

- 在超弹性模型基础上，引入不可恢复形变和时间相关行为。
- 弹塑性分解：

$$\mathbf{F} = \mathbf{F}_e \mathbf{F}_p$$

- 粘弹性（最简形式）：

$$\mathbf{P} = \mathbf{P}_{elastic} + \eta \dot{\mathbf{F}}$$

- 最小实现:

- 每个单元存储 F_p （初始为单位阵）
- 额外存储上一帧 F^{prev} （用于粘性项）
- 每帧：

a. 计算当前 F

b. SVD 分解： $F = U\Sigma V^T$

c. 裁剪奇异值：

$$\sigma_i^{new} = \min(\sigma_i, \sigma_{max})$$

d. 更新塑性部分：

$$\mathbf{F}_p \leftarrow V\Sigma_{new}V^T$$

e. 有限差分计算粘性项：

$$\dot{\mathbf{F}} \approx \frac{\mathbf{F} - \mathbf{F}^{prev}}{\Delta t}$$

f. 总应力：

$$\mathbf{P} = \mathbf{P}_{elastic}(F_e) + \eta \dot{\mathbf{F}}$$

g. 力更新速度，保存 $F^{prev} \leftarrow F$

- 参数建议
 - $\eta = 0 \rightarrow$ 无粘性（退化为普通弹性）
 - $\eta \in [0.01, 1] \rightarrow$ 可观察到阻尼/松弛效果
 - η 越大 $\rightarrow$ 越“粘”（像橡皮泥）
- 效果：
 - 拉伸后不能完全恢复（弹塑性）
 - 运动过程中有明显阻尼（粘性）
 - 保持形变时“慢慢稳定下来”

建议与提示

0. 写在开头

- 本次 Lab 要求实现的内容核心在于给定四面体（三角形）的形变后，如何计算受力。所以在动手开始写代码前，十分推荐你首先将这一部分的数学仔细地推导一遍。具体来说，我们知道单位体积（面积）受到的力 $\mathbf{f}$ 可以被表示为：

$$\mathbf{f} = \frac{\partial \Psi}{\partial \mathbf{x}} = \frac{\partial \Psi}{\partial F} \frac{\partial F}{\partial \mathbf{x}}.$$

这其中的 Ψ 为我们选定的能量形式， F 为 Deformation Gradient， $\mathbf{x}$ 为位移。 $\frac{\partial \Psi}{\partial F}$ 即我们熟知的 First Piola-Kirchhoff stress P ，它是一个矩阵；而剩下的 $\frac{\partial F}{\partial \mathbf{x}}$ 则是一个三阶张量。**如何使用 cpp 代码实现矩阵与三阶张量的相乘，这是你开始动手前需要思考清楚的问题。** 具体的，你可以根据课上所学，并结合文档最后罗列的[参考资料](#)，找到一种你觉得合适的实现方案。

- 在开始 Demo 中展示的大场景模拟之前，我们推荐首先在一些小场景中测试你的代码。如单独的一个四面体（三角形），这将为你 Debug 提供许多便利。
- 关于选做的布料模拟，逻辑与软体模拟类似。只是组成布料的三角形在 Rest Shape 下只需要二维坐标就可以描述，相应得到的 F 将是一个 3×2 的矩阵，而不是 3×3 的方阵，这会使其与软体模拟略有不同。如果觉得数学的推导过于复杂，也可以参考[论文](#)进行实现。

1. 参数设置

- 为了更快更好地得到稳定的模拟结果，在软体模拟中，我们推荐的一组相对稳定的参数为杨氏模量取 20000 Pa，泊松比取 0.2，弹性体的密度为 400 kg/m^3 ，重力大小为 0.05 m/s^2 ，模拟软体的长宽高为 $8\text{m} \times 2\text{m} \times 2\text{m}$ ；而在布料模拟中，我们推荐的一组相对稳定的参数为杨氏模量取 50 Pa，泊松比取 0.3，布料的面密度为 0.5 kg/m^2 ，重力的大小为 9.8 m/s^2 ，模拟布料的长宽为 $2\text{m} \times 2\text{m}$ 。同时由于我们采用的是显式模拟的方法，可以通过适当调小时间步长（大约至 0.001 s 左右），并为模拟结果添加一些阻尼来提高稳定性。

2. 初始化

- 关于长方体的四面体网格初始化可以[参考](#)如下方式，首先我们用正方体网格切分长方体，之后将用网格的各个顶点作为四面体网格的顶点：

```
for (std::size_t i = 0; i <= _tetSystem.wx; i++) {
    for (std::size_t j = 0; j <= _tetSystem.wy; j++) {
        for (std::size_t k = 0; k <= _tetSystem.wz; k++) {
            _tetSystem.AddParticle({ i * delta, j * delta, k * delta});
        }
    }
}
```

其中的 `_tetSystem.wx`，`_tetSystem.wy` 和 `_tetSystem.wz` 分别代表长方体在 `xyz` 三个方向上网格的数量，`delta` 代表网格的间距。`_tetSystem.AddParticle` 会维护一个顶点向量 `p`，依次添加每个顶点。这样我们可以通过下方的 `_tetSystem.GetID` 函数在输入顶点的 `(i,j,k)` 坐标后得到其在顶点向量 `p` 中的位置，也就是顶点的 `ID`。

```
inline int GetID(std::size_t const i, std::size_t const j, std::size_t const k) {
    return i * (wy + 1) * (wz + 1) + j * (wz + 1) + k;
}
```

最后利用如下代码，我们将通过 `_tetSystem.AddTet` 函数，依次记录每个四面体四个顶点的 `ID`，从而完成长方体的四面体网格初始化。

```

for (std::size_t i = 0; i < _tetSystem.wx; i++) {
    for (std::size_t j = 0; j < _tetSystem.wy; j++) {
        for (std::size_t k = 0; k < _tetSystem.wz; k++) {
            _tetSystem.AddTet(_tetSystem.GetID(i, j, k),
                _tetSystem.GetID(i, j, k + 1),
                _tetSystem.GetID(i, j + 1, k + 1),
                _tetSystem.GetID(i + 1, j + 1, k + 1));
            _tetSystem.AddTet(_tetSystem.GetID(i, j, k),
                _tetSystem.GetID(i, j + 1, k),
                _tetSystem.GetID(i, j + 1, k + 1),
                _tetSystem.GetID(i + 1, j + 1, k + 1));
            _tetSystem.AddTet(_tetSystem.GetID(i, j, k),
                _tetSystem.GetID(i, j, k + 1),
                _tetSystem.GetID(i + 1, j, k + 1),
                _tetSystem.GetID(i + 1, j + 1, k + 1));
            _tetSystem.AddTet(_tetSystem.GetID(i, j, k),
                _tetSystem.GetID(i + 1, j, k),
                _tetSystem.GetID(i + 1, j, k + 1),
                _tetSystem.GetID(i + 1, j + 1, k + 1));
            _tetSystem.AddTet(_tetSystem.GetID(i, j, k),
                _tetSystem.GetID(i, j + 1, k),
                _tetSystem.GetID(i + 1, j + 1, k),
                _tetSystem.GetID(i + 1, j + 1, k + 1));
            _tetSystem.AddTet(_tetSystem.GetID(i, j, k),
                _tetSystem.GetID(i + 1, j, k),
                _tetSystem.GetID(i + 1, j + 1, k),
                _tetSystem.GetID(i + 1, j + 1, k + 1));
        }
    }
}

```

3. 用户交互

- 我们推荐通过控制模拟对象的某一个点来实现交互。具体来说，你可以通过给某一个点施加一个正比于鼠标移动距离的力来使这个点发生位移，关于鼠标移动距离的获取可以参考 Lab0 中

`CaseBox::OnProcessMouseControl` 提供的简单示例。而关于如何切换控制不同的点，一种相对简单的实现方式为课上展示的，通过给定模拟对象 `Rest Shape` 下的顶点坐标实现。当然，你也可以选择实现更高级的切换方式，例如每次查找在屏幕空间中距离鼠标位置最近的点作为控制点等等。

作业提交

作业须提交到[教学网](#)。

请确保你使用 `Git` 管理你的代码，在完成 Lab3 时输出你的提交记录：

```
git log --stat > lab3_log.txt
```

VCX 用户请执行 `xmake clean -a`；Taichi 用户请删除 `pycache` 等临时文件夹。

同时，也请提交一份作业报告，简要说明核心代码的思路，实现的交互方法，并展示你的 Demo。

将你输出的提交记录、源代码和报告打包成 `zip` 压缩包，并以 `lab3_<your-student-ID>.zip` 命名，提交到教学网。

参考资料

- [FEM simulation of 3D deformable solids: a practitioner's guide to theory, discretization and model reduction](#): Siggraph 2012 课程，其 Part One 详细地介绍了 FEM 的理论模型和离散化方案。
- [Dynamic deformables: implementation and production practicalities \(now with code!\)](#): Siggraph 2022 课程，在第 2-4 章详细介绍了 FEM 中常用的关于能量的一阶导数、二阶导数（隐式 FEM 会用到）的求导过程以及代码实现。
- [GAMES103](#)：优质线上课程，在其 Lecture 07 和 Lecture 08 中介绍了基于 FEM 的软体模拟。
- [Physics Simulation in Visual Computing](#)：使用 Javascript 实现了多种能量形式的二维 FEM 模拟。